JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class StringTokenizer

java.lang.Object
    java.util.StringTokenizer

All Implemented Interfaces:

Enumeration<Object>

```
public class StringTokenizer
extends Object
implements Enumeration<Object>
```

The string tokenizer class allows an application to break a string into tokens. The tokenization method is much simpler than the one used by the `StreamTokenizer` class. The `StringTokenizer` methods do not distinguish among identifiers, numbers, and quoted strings, nor do they recognize and skip comments.

The set of delimiters (the characters that separate tokens) may be specified either at creation time or on a per-token basis.

An instance of `StringTokenizer` behaves in one of two ways, depending on whether it was created with the `returnDelims` flag having the value `true` or `false`:

- If the flag is `false`, delimiter characters serve to separate tokens. A token is a maximal sequence of consecutive characters that are not delimiters.
- If the flag is `true`, delimiter characters are themselves considered to be tokens. A token is thus either one delimiter character, or a maximal sequence of consecutive characters that are not delimiters.

A `StringTokenizer` object internally maintains a current position within the string to be tokenized. Some operations advance this current position past the characters processed.

A token is returned by taking a substring of the string that was used to create the `StringTokenizer` object.

The following is one example of the use of the tokenizer. The code:

```
StringTokenizer st = new StringTokenizer("this is a test");
while (st.hasMoreTokens()) {
    System.out.println(st.nextToken());
}
```

prints the following output:

```
this
is
a
test
```

`StringTokenizer` is a legacy class that is retained for compatibility reasons although its use is discouraged in new code. It is recommended that anyone seeking this functionality use the `split` method of `String` or the java.util.regex package instead.

The following example illustrates how the `String.split` method can be used to break up a string into its basic tokens:

```
String[] result = "this is a test".split("\\s");
for (int x=0; x<result.length; x++)
    System.out.println(result[x]);
```

prints the following output:

```
this
is
a
test
```

Since:

JDK1.0

See Also:

`StreamTokenizer`

Constructor Summary

Constructors

Constructor	Description

`StringTokenizer(String str)`	Constructs a string tokenizer for the specified string.
`StringTokenizer(String str, String delim)`	Constructs a string tokenizer for the specified string.
`StringTokenizer(String str, String delim, boolean returnDelims)`	Constructs a string tokenizer for the specified string.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`int`	`countTokens()`	Calculates the number of times that this tokenizer's `nextToken` method can be called before it generates an exception.
`boolean`	`hasMoreElements()`	Returns the same value as the `hasMoreTokens` method.
`boolean`	`hasMoreTokens()`	Tests if there are more tokens available from this tokenizer's string.
`Object`	`nextElement()`	Returns the same value as the `nextToken` method, except that its declared return value is `Object` rather than `String`.
`String`	`nextToken()`	

		Returns the next token from this string tokenizer.
String	nextToken(String delim)	Returns the next token in this string tokenizer's string.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

StringTokenizer

```
public StringTokenizer(String str,
                       String delim,
                       boolean returnDelims)
```

Constructs a string tokenizer for the specified string. All characters in the `delim` argument are the delimiters for separating tokens.

If the `returnDelims` flag is `true`, then the delimiter characters are also returned as tokens. Each delimiter is returned as a string of length one. If the flag is `false`, the delimiter characters are skipped and only serve as separators between tokens.

Note that if `delim` is `null`, this constructor does not throw an exception. However, trying to invoke other methods on the resulting `StringTokenizer` may result in a `NullPointerException`.

Parameters:

`str - a string to be parsed.`

`delim - the delimiters.`

`returnDelims - flag indicating whether to return the delimiters as tokens.`

Throws:

`NullPointerException - if str is null`

StringTokenizer

```
public StringTokenizer(String str,
                       String delim)
```

Constructs a string tokenizer for the specified string. The characters in the `delim` argument are the delimiters for separating tokens. Delimiter characters themselves will not be treated as tokens.

Note that if `delim` is `null`, this constructor does not throw an exception. However, trying to invoke other methods on the resulting `StringTokenizer` may result in a `NullPointerException`.

Parameters:

`str - a string to be parsed.`

`delim - the delimiters.`

Throws:

`NullPointerException - if str is null`

StringTokenizer

```
public StringTokenizer(String str)
```

Constructs a string tokenizer for the specified string. The tokenizer uses the default delimiter set, which is `" \t\n\r\f"`: the space character, the tab character, the newline character, the carriage-return character, and the form-feed character. Delimiter characters themselves will not be treated as tokens.

Parameters:

`str - a string to be parsed.`

Throws:

`NullPointerException - if str is null`

Method Detail

hasMoreTokens

```
public boolean hasMoreTokens()
```

Tests if there are more tokens available from this tokenizer's string. If this method returns `true`, then a subsequent call to `nextToken` with no argument will successfully return a token.

Returns:

`true` if and only if there is at least one token in the string after the current position; `false` otherwise.

nextToken

```
public String nextToken()
```

Returns the next token from this string tokenizer.

Returns:

the next token from this string tokenizer.

Throws:

`NoSuchElementException` - if there are no more tokens in this tokenizer's string.

nextToken

```
public String nextToken(String delim)
```

Returns the next token in this string tokenizer's string. First, the set of characters considered to be delimiters by this `StringTokenizer` object is changed to be the characters in the string `delim`. Then the next token in the string after the current position is returned. The current position is advanced beyond the recognized token. The new delimiter set remains the default after this call.

Parameters:

`delim` - the new delimiters.

Returns:

the next token, after switching to the new delimiter set.

Throws:

`NoSuchElementException` - if there are no more tokens in this tokenizer's string.

`NullPointerException` - if delim is null

hasMoreElements

```
public boolean hasMoreElements()
```

Returns the same value as the `hasMoreTokens` method. It exists so that this class can implement the `Enumeration` interface.

Specified by:

`hasMoreElements` in interface `Enumeration<Object>`

Returns:

`true if there are more tokens; false otherwise.`

See Also:

`Enumeration, hasMoreTokens()`

nextElement

```
public Object nextElement()
```

Returns the same value as the `nextToken` method, except that its declared return value is `Object` rather than `String`. It exists so that this class can implement the `Enumeration` interface.

Specified by:

`nextElement in interface Enumeration<Object>`

Returns:

`the next token in the string.`

Throws:

`NoSuchElementException - if there are no more tokens in this tokenizer's string.`

See Also:

`Enumeration, nextToken()`

countTokens

```
public int countTokens()
```

Calculates the number of times that this tokenizer's `nextToken` method can be called before it generates an exception. The current position is not advanced.

Returns:

`the number of tokens remaining in the string using the current delimiter set.`

See Also:

```
nextToken()
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.